

Fraud Detection System

Model Workflow, Feature Guide & Evaluation Results

A plain-language walkthrough of the dataset, the modeling decisions, and what the results mean — written so it can be turned directly into a project report.

1. What This System Does

This project is a fraud detection system for bank card transactions. Given the details of a transaction (amount, merchant, channel, device, and a set of behavioural signals), a trained machine learning model produces a **risk score** — a percentage estimate of how likely that transaction is to be fraudulent. Transactions above a chosen risk threshold are flagged for a staff member to review and decide on (Approve, Investigate, or Reject).

The system does not attempt to auto-block transactions. This is a deliberate design choice, explained in Section 6: the cost of missing real fraud is far higher than the cost of a staff member spending a few minutes reviewing a false alarm, so the model is intentionally tuned to catch more fraud even if that means more transactions get flagged for review.

Why This Is a Hard Problem

Fraud is rare. In this dataset, only 1.7% of transactions are fraudulent — the other 98.3% are completely normal. A model that just guessed “legitimate” every single time would be 98.3% accurate and completely useless, since it would catch zero fraud. This is why the project does not use plain accuracy to judge the model (Section 5 explains this in detail) and why specific techniques (SMOTE, threshold tuning) are used to handle this imbalance.

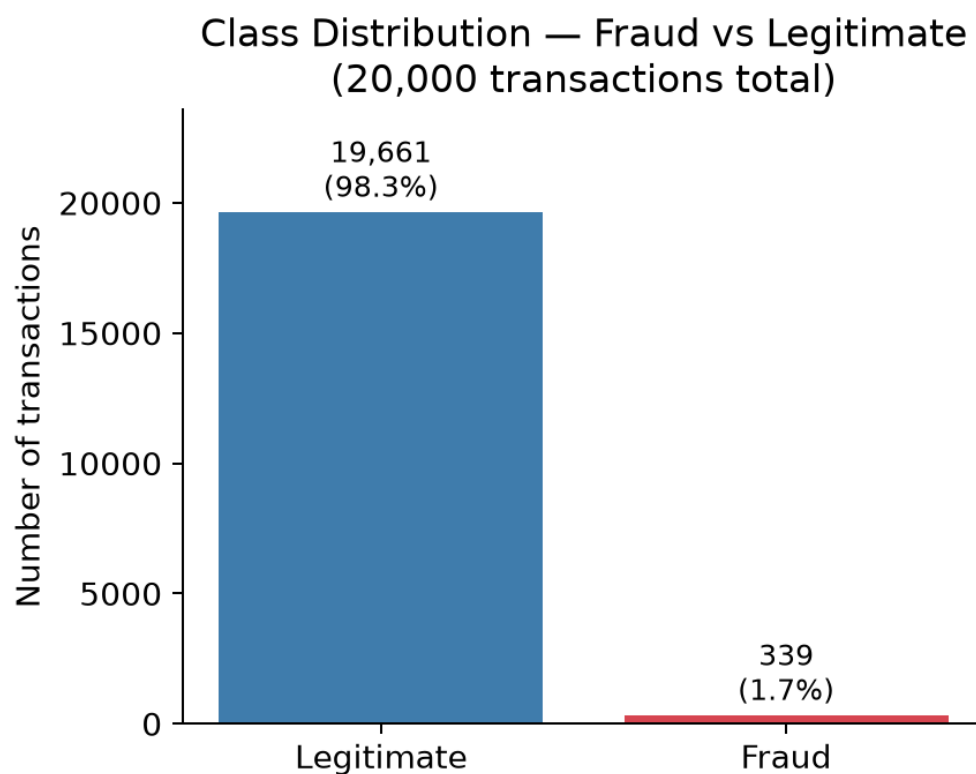


Figure 1 — Class balance in the dataset: fraud is a small minority, which is realistic (real-world card fraud is often even rarer) but makes the model harder to train and evaluate fairly.

2. Understanding the Data: What Each Column Means

Every transaction is described by a set of columns (features). Some describe the transaction itself, some describe the customer's account, and some are calculated automatically from recent activity. The table below explains each one in plain terms and why it plausibly relates to fraud. *Note: in the actual dataset and code, these column names use underscores instead of spaces (e.g. "distance from home km" appears in code as `distance_from_home_km`) — shown with spaces here for readability.*

Transaction details

Column	What it represents	Why it can signal fraud
amount ngn	The Naira value of the transaction.	Unusually large amounts (relative to what's normal) are a classic fraud signal.
merchant category	The type of business the payment was made to (e.g. Restaurants, Crypto Exchange, Gaming/Betting).	Some merchant types are inherently higher-risk for fraud/money-laundering than others (e.g. crypto exchanges).
channel	How the transaction was made: Internet Banking, POS, Mobile App, USSD, or ATM.	Some channels have weaker identity checks than others, making them easier to exploit.
device type	The device used (Android Phone, iPhone, Windows PC, POS Terminal, ATM Machine, etc.).	An unfamiliar or unusual device for a given channel can indicate account takeover.
auth method	How the transaction was authenticated: OTP, 3D Secure, PIN, Biometric, or No Authentication.	Weaker authentication (especially 'No Authentication') removes a key barrier fraudsters must bypass.
cvv retry count	How many times the card's security code was mistyped before this transaction.	Repeated CVV failures often mean someone is guessing the code, a sign of card-not-present fraud.

Location & timing

Column	What it represents	Why it can signal fraud
distance from home km	How far the transaction location is from the customer's home.	A transaction far from where the customer normally is can indicate a stolen card.
is foreign transaction	Whether the transaction happened outside the customer's home country.	Cross-border transactions are statistically more likely to be fraudulent.
time of day hour / day of week	When the transaction happened.	Fraud disproportionately happens at unusual hours (e.g. late at night) when a legitimate card owner is unlikely to be transacting.

Account & customer profile

Column	What it represents	Why it can signal fraud
card type	Visa, Mastercard, or Verve.	Card networks differ in adoption and fraud-prevention tooling, giving each a slightly different baseline risk.
card age months	How long the card has existed.	Newly issued cards are more often targeted or used in fraud rings than long-established ones.
customer age	The account holder's age.	Certain age groups are statistically more or less likely to be fraud targets or victims.
account balance ngn	The customer's current account balance.	Balance relative to transaction size can indicate whether a purchase is plausible for that account.
prior disputes	How many times this customer has previously disputed a transaction.	A history of disputes can mean the account is a repeat fraud target or otherwise higher-risk.

Behavioural signals (calculated automatically)

Column	What it represents	Why it can signal fraud
hours since last txn	Time gap since this customer's previous transaction.	Fraud often comes in rapid bursts once a card is compromised — a very short gap is suspicious.
txn count last 24h	How many transactions this customer has made in the last day.	An unusually high transaction count in a short time is a hallmark of card testing/fraud rings.
velocity score	A single number summarizing how fast this account has been transacting recently.	Combines the two signals above into one strong, easy-to-use fraud indicator (see Section 6 — it's one of the two most important features).
merchant risk score	The average historical fraud rate for this merchant category.	Some categories (e.g. Crypto Exchange) are consistently riskier — this bakes that pattern directly into the model (the single most important feature).

Explicit risk flags

Column	What it represents	Why it can signal fraud
is new merchant	Whether this is the customer's first payment to this merchant.	New payees are a common early sign of a compromised account being drained.
used vpn	Whether a VPN was detected.	Fraudsters use VPNs to mask their real location.
ip country mismatch	Whether the connection's IP address country differs from the card's country.	A strong signal of remote/unauthorized access.

Column	What it represents	Why it can signal fraud
billing shipping mismatch	Whether billing and shipping addresses differ.	A classic online-fraud pattern — goods shipped somewhere other than the cardholder's address.
is ai generated scam attempt	Whether the transaction shows signs of an AI-generated scam (e.g. deepfake-assisted social engineering).	A modern, emerging fraud vector worth tracking explicitly.

3. Preparing the Data

Adapting the dataset

The starting dataset is a Kaggle credit-card-fraud dataset using US terms (USD amounts, generic channel names, international card schemes). Before training, every category is remapped to a Nigerian banking context — amounts converted to Naira, channels renamed to Internet Banking/USSD/Mobile App/POS/ATM, and cards consolidated to Visa/Mastercard/Verve. This remapping happens *before* the model is trained, so the model genuinely learns on these categories rather than just having them relabeled afterwards for display.

Turning raw data into model-ready numbers

Machine learning models need numbers, not text. Categories like “merchant_category” or “auth_method” are converted to numbers through a process called **label encoding** (each distinct category gets assigned an integer code, e.g. “Visa” = 0, “Mastercard” = 1). True/False flags are converted to 1/0. The transaction amount is also log-transformed (a standard technique that stops a few very large transactions from dominating the model's view of what's “normal”), and a simple “is it night-time?” flag is derived from the hour of day.

Splitting the data fairly

The data is split 80% for training and 20% for testing, using a **stratified split** — meaning both the training set and the test set keep the same 1.7% fraud ratio. This matters because a careless random split could accidentally put almost no fraud cases in the test set, making the evaluation meaningless.

Fixing the class imbalance: SMOTE

What is SMOTE? Short for Synthetic Minority Oversampling Technique. Since fraud cases are rare, a model trained directly on the raw data would barely notice them and mostly learn to predict “legitimate”. SMOTE creates additional, synthetic (but realistic) fraud examples by interpolating between existing real fraud cases, until the training set has a roughly equal number of fraud and non-fraud examples. Crucially, **SMOTE is applied only to the training data** — the test set is left at its real, untouched 1.7% fraud rate, so the reported results reflect real-world conditions rather than an artificially easy, balanced test.

4. Choosing the Model

Two models were trained and compared: **Random Forest** and **XGBoost**. Both are “ensemble” methods that work by building many decision trees (a decision tree is a series of simple yes/no questions, like “Is the merchant risk score above 50? → Is the authentication method 'No Authentication'? → ...”) and combining their votes into a final answer. This family of model was chosen over deep learning because on structured, spreadsheet-style data like this, tree ensembles consistently perform as well or better, train in seconds rather than hours, and are far easier to interpret and explain — a real advantage when the results need to be defended and understood by people who aren't machine learning specialists.



Figure 2 — Random Forest scored a meaningfully higher ROC-AUC than XGBoost on the same held-out test set (0.753 vs 0.685), so it was selected as the deployed model.

Why ROC-AUC and not accuracy to compare them? As explained in Section 1, accuracy is misleading on this dataset. ROC-AUC (explained fully in Section 5) measures how well a model ranks fraud above legitimate transactions, regardless of any specific cutoff point, which makes it a fair way to compare two models.

5. Evaluation Metrics, Explained Simply

This section explains each evaluation term used in this project in plain language, before Section 6 shows what these numbers actually turned out to be for this model.

The Confusion Matrix

Every prediction the model makes falls into one of four buckets, usually drawn as a 2×2 grid:

	Model said: Legitimate	Model said: Fraud
Actually Legitimate	True Negative (correct)	False Positive (false alarm)
Actually Fraud	False Negative (missed fraud)	True Positive (correctly caught)

Every other metric below is just a different way of summarizing these four numbers.

Accuracy — and why it's misleading here

Accuracy = (correct predictions) ÷ (all predictions). It sounds like the obvious choice, but on a dataset that's 98.3% legitimate, a model can score 98.3% accuracy by simply never flagging anything as fraud — a completely useless model. This is why accuracy is not used to judge this model's performance.

Precision — “of everything we flagged, how much was real fraud?”

Precision = True Positives ÷ (True Positives + False Positives). A high precision means that when the system raises an alarm, it's usually right. Low precision means lots of false alarms — costly in staff time, but not dangerous, since nothing gets auto-blocked.

Recall — “of all the real fraud, how much did we actually catch?”

Recall = True Positives ÷ (True Positives + False Negatives). A high recall means the model misses very little real fraud. Low recall means a lot of fraud slips through undetected — the costly, dangerous kind of mistake in this context.

F1-score — a single balance of precision and recall

F1 is the harmonic mean of precision and recall — a single number that's only high when both precision and recall are reasonably good. It's useful for comparing models generally, but it treats “missed fraud” and “false alarm” as equally bad, which (as Section 6 explains) is not true for this system.

ROC-AUC — how well the model ranks risk overall

ROC-AUC (“Area Under the ROC Curve”) measures how well the model separates fraud from legitimate transactions across every possible cutoff point, not just one. A score of 0.5 means the model is no better than random guessing; 1.0 would mean perfect separation. Because it doesn't depend on picking one specific threshold, it's the fairest single number for comparing two models, which is why it was used to choose between Random Forest and XGBoost in Section 4.

6. Our Results, Explained

How well the model separates fraud from legitimate transactions

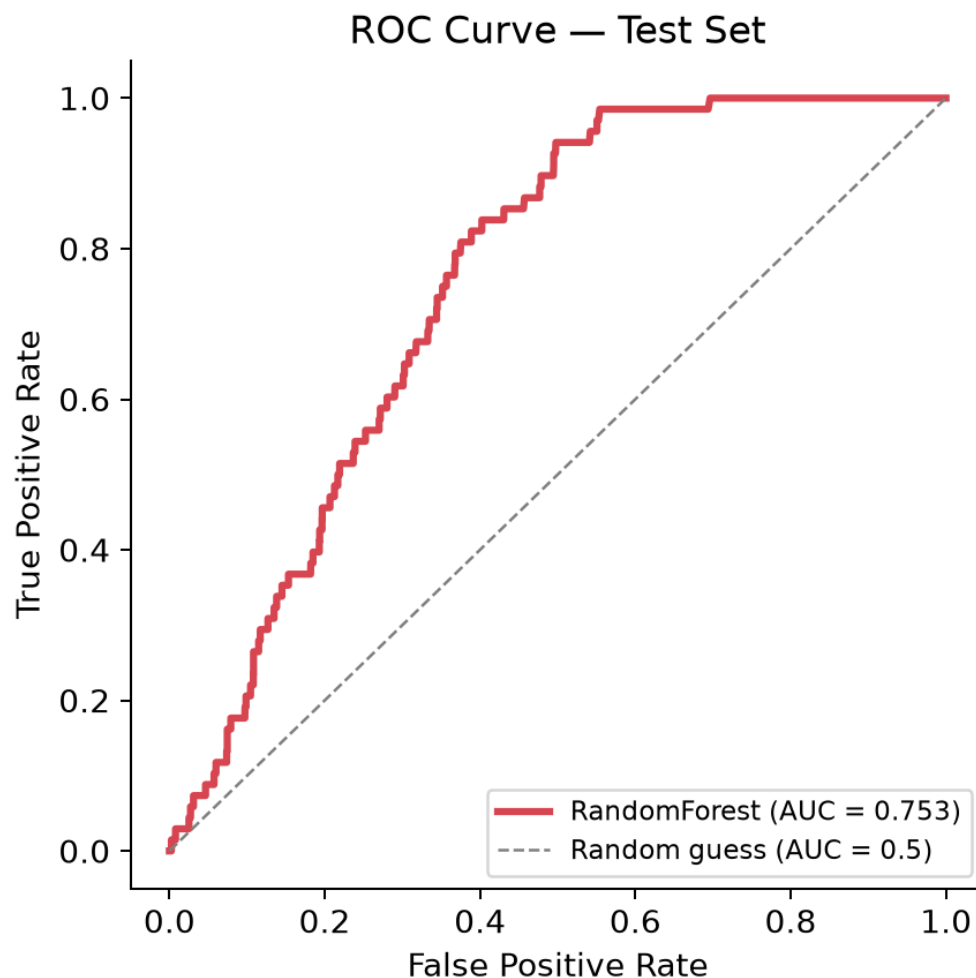


Figure 3 — ROC curve for the selected Random Forest model. $AUC = 0.753$, meaning the model does a genuinely meaningful (though imperfect) job of ranking fraud above legitimate transactions — a random model would trace the diagonal dashed line ($AUC = 0.5$).

Choosing the operating threshold

A model outputs a risk score from 0% to 100% for every transaction — it doesn't decide “fraud” or “not fraud” on its own. A cutoff (threshold) has to be chosen: any transaction scored above it gets flagged. Picking this cutoff is a genuine business decision, not a purely technical one, because of the trade-off shown below.

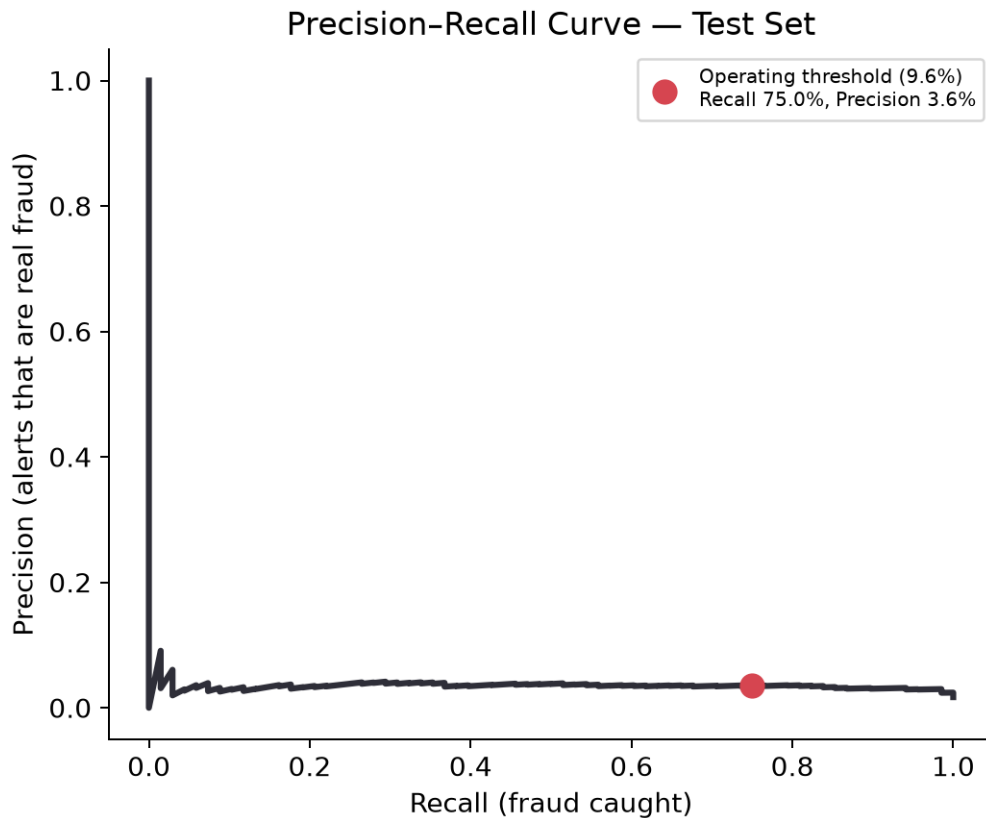


Figure 4 — Precision vs. recall at every possible threshold. The marked point is the threshold actually used: 9.6%, chosen to guarantee catching about 75% of fraud, accepting a low precision (3.6%) as the cost of that.

Why prioritize recall over precision here? In this system, a flagged transaction is not auto-blocked — it goes to a staff member to review (Approve / Investigate / Reject). A false alarm therefore costs a few minutes of staff time. A missed fraud case costs the bank real money. Given that imbalance in consequences, the threshold was deliberately chosen to catch more fraud (higher recall) even though it means far more transactions get flagged for review (lower precision) than a “balanced” threshold would produce. This is a considered trade-off, not an oversight — it is worth stating explicitly in a project report as evidence of understanding the real-world cost asymmetry in fraud detection.

The result, laid out plainly

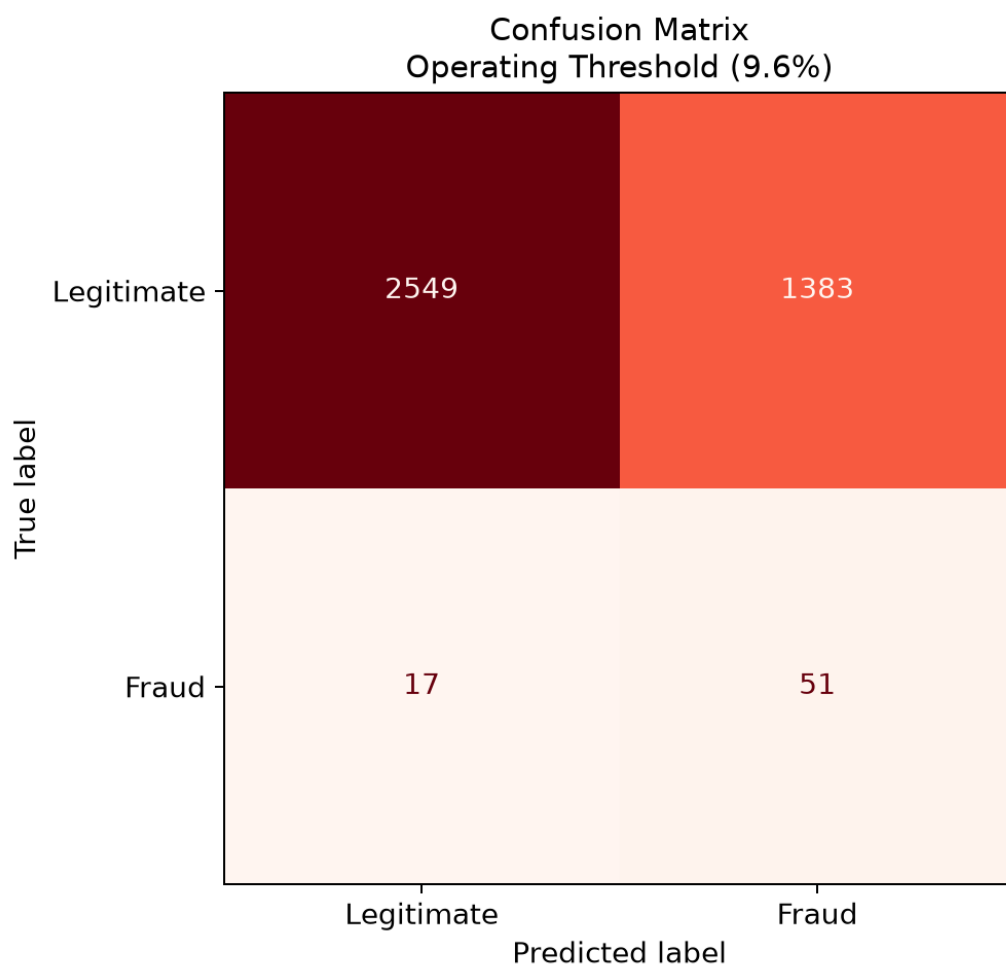


Figure 5 — Confusion matrix on the test set at the chosen 9.6% threshold.

Outcome	Count	What it means
True Positives	51	Real fraud cases correctly flagged for review.
False Negatives	17	Real fraud cases the model missed (25% of all fraud in the test set).
False Positives	1,383	Legitimate transactions incorrectly flagged (cost: staff review time).
True Negatives	2,549	Legitimate transactions correctly left alone.

In plain terms: out of 68 real fraud cases in the test set, the system caught 51 of them (75% recall). To achieve that, it also flagged 1,383 legitimate transactions that turned out to be fine — meaning only about 1 in 28 flagged transactions is genuinely fraudulent (3.6% precision). That sounds low in isolation, but is expected and considered acceptable in fraud detection specifically because of the review workflow described above — this is a widely recognized, realistic trade-off, not a flaw unique to this project.

What the model actually learned to pay attention to

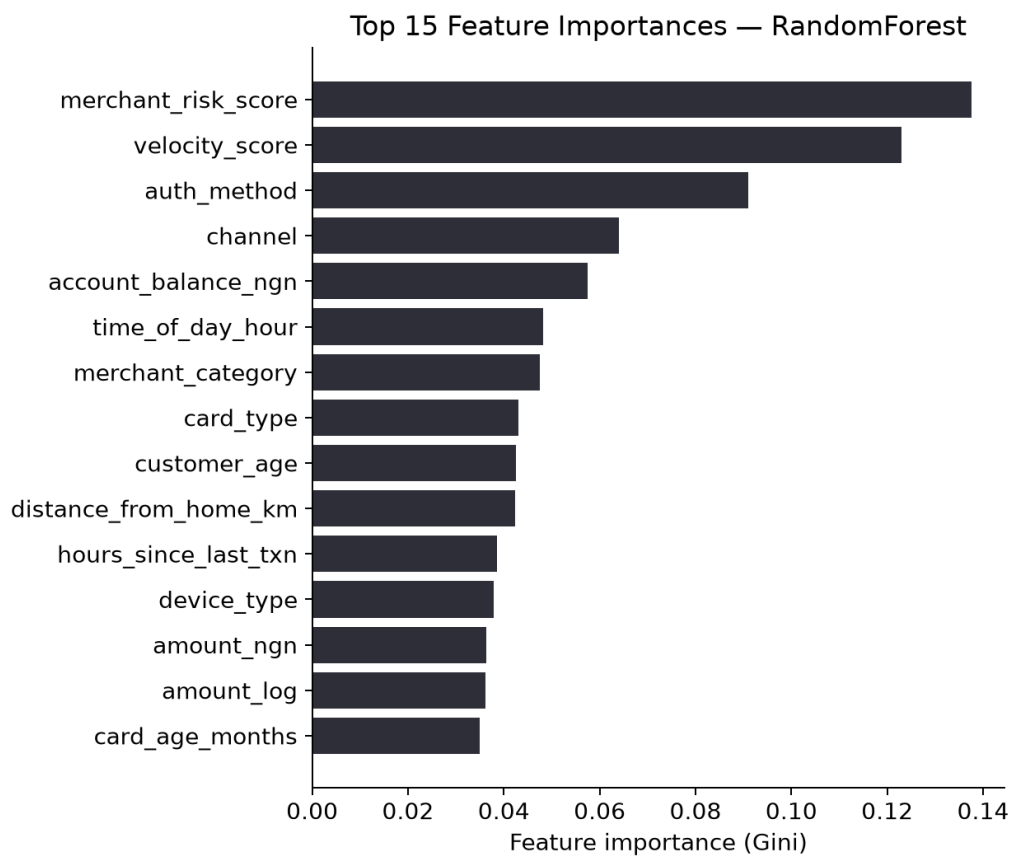


Figure 6 — The features the model relied on most. *merchant_risk_score* and *velocity_score* dominate, followed by *auth_method* — all of which line up with well-known fraud indicators, supporting that the model learned genuine patterns rather than noise.

7. How It All Fits Together

The diagrams below summarize the two halves of the project: the offline process that produces the trained model, and the live application that uses it to score real transactions.

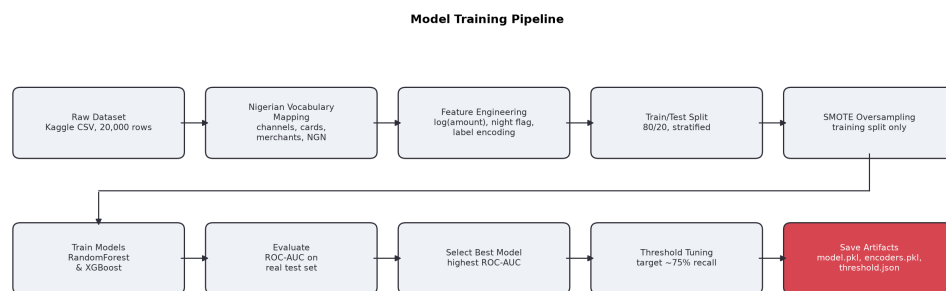


Figure 7 — The model training pipeline, start to finish.

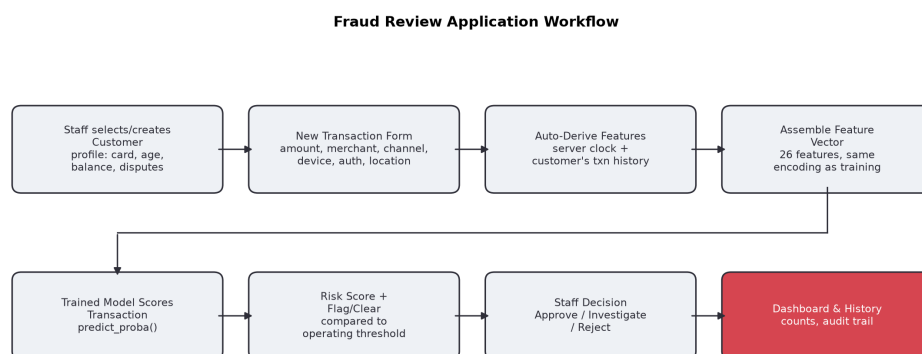


Figure 8 — What happens when a staff member submits a transaction in the live application.

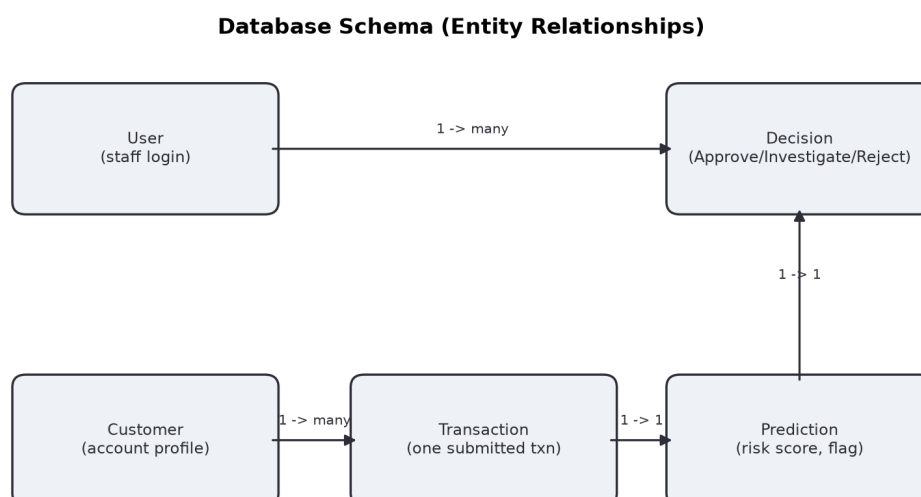


Figure 9 — How the application's data is structured.

8. Limitations & Honest Caveats

A good project report states its limitations clearly rather than hiding them — examiners and readers generally respond better to honesty here than to an implied claim of perfection.

Column	What it represents	Why it can signal fraud
Synthetic data	The dataset is Kaggle-generated, not real bank transaction logs (real data is not accessible for a project like this).	Real-world patterns may differ; results should be read as a demonstration of method, not a production benchmark.
Fraud rate	1.7% fraud is higher than real card fraud rates (often well under 0.5%).	A more imbalanced real dataset would likely be even harder to achieve high precision on.
velocity score formula	In the live app, this is computed with a simple hand-written formula from recent transaction count and recency.	A production system would derive this from much richer historical behavioural data.
Low precision in absolute terms	Only ~3.6% of flagged transactions are truly fraudulent.	Acceptable given the staff-review workflow, but a larger dataset and richer features (e.g. network/graph-based signals between accounts) would be the natural next step to improve this.

9. Conclusion

The trained Random Forest model shows genuine, meaningful ability to separate fraudulent from legitimate transactions (ROC-AUC 0.753), built on a dataset deliberately adapted to reflect a realistic class imbalance and a Nigerian banking context. Rather than using a default decision threshold, the operating point was deliberately chosen to catch about 75% of fraud, reflecting a considered judgment about the relative costs of missed fraud versus false alarms in a system where every flag goes to human review. The features the model relies on most — merchant risk, transaction velocity, and authentication strength — align with recognized fraud indicators, supporting that the model has learned real, defensible patterns.